

Przewodnik egzaminacyjny Java — Część 2: Rekordy (pytania 41-60)

Rekordy w Javie — wprowadzenie

Rekordy (`record`) zostały wprowadzone w **Java 16** jako standardowa funkcja (wcześniej w preview od Java 14). Są to specjalne klasy zaprojektowane do przechowywania niemutowalnych danych. Kompilator automatycznie generuje za nas cały "boilerplate".

```
java

// Zamiast pisać klasę z polami, konstruktorem, getterami, equals, hashCode, toString
record Point(int x, int y) { }

// Automatycznie generowane przez kompilator:
// - konstruktor kanoniczny: Point(int x, int y)
// - akcesory: x(), y() (NIE getX(), getY())
// - equals(), hashCode(), toString()
// - klasa jest final (nie można po niej dziedziczyć)
```

41. Główny cel rekordów

Cel: Rekordy to **nośniki danych** (data carriers) — klasy, których jedynym zadaniem jest przechowywanie niezmiennego zestawu wartości. Eliminują boilerplate (powtarzalny kod).

Bez rekordów:

```
java
```

```
public final class Point {  
    private final int x;  
    private final int y;  
  
    public Point(int x, int y) { this.x = x; this.y = y; }  
    public int x() { return x; }  
    public int y() { return y; }  
  
    @Override  
    public boolean equals(Object o) { ... } // 5+ linii  
    @Override  
    public int hashCode() { ... }  
    @Override  
    public String toString() { ... }  
}
```

Z rekordem:

```
java  
  
record Point(int x, int y) { } // WSZYSTKO powyżej w jednej linii!
```

42. Walidacja w konstruktorze kompaktowym

Rekordy mają specjalny **konstruktor kompaktowy** (compact constructor) do walidacji:

```
java
```

```

record Range(int min, int max) {
    // Konstruktor kompaktowy – brak listy parametrów!
    Range {
        // Parametry są dostępne pod nazwami komponentów
        if (min > max) {
            throw new IllegalArgumentException("min > max: " + min + " > " + max);
        }
        // Normalizacja – możliwa!
        // min i max są przypisywane niejawnie po tym bloku
    }
}

// Standardowy konstruktor też jest możliwy:
record Person(String name, int age) {
    Person {
        Objects.requireNonNull(name, "Name cannot be null");
        if (age < 0) throw new IllegalArgumentException("Age cannot be negative");
        name = name.trim(); // normalizacja – kompilator przypisze tę wartość
    }
}

```

Kluczowe: W konstruktorze kompaktowym przypisanie do komponentów jest **niejawne** — dzieje się automatycznie na końcu bloku. Możesz jednak nadpisać wartość przed tym (normalizacja).

43. Czy rekord może rozszerzać inną klasę?

NIE. Rekord niejawnie rozszerza (`java.lang.Record`). Java nie pozwala na wielokrotne dziedziczenie klas → rekord nie może rozszerzać żadnej innej klasy.

```

java

class Base { }

// ❌ BŁĄD KOMPILACJI
record MyRecord(int x) extends Base { }

// ✅ Rekordy mogą implementować interfejsy
interface Printable { void print(); }
record Point(int x, int y) implements Printable {
    @Override
    public void print() { System.out.println("(" + x + ", " + y + ")"); }
}

```

44. Rekordy jako klucze w HashMap

Tak, to bezpieczne. Rekordy automatycznie generują `equals()` i `hashCode()` na podstawie wszystkich komponentów. To podstawowy warunek bezpiecznego użycia jako kluczy w `HashMap`.

```
java

record Coordinate(int lat, int lon) { }

Map<Coordinate, String> map = new HashMap<>();
Coordinate c1 = new Coordinate(52, 21);
Coordinate c2 = new Coordinate(52, 21);

map.put(c1, "Warsaw");
System.out.println(map.get(c2)); // ✅ "Warsaw" – c1.equals(c2) = true, hashCode r
```

Dlaczego bezpieczne: Rekordy są niemutowalne (komponenty są `private final`). Klucz `HashMap` nie zmieni swojego `hashCode()` po wstawieniu, bo nie można zmienić wartości komponentów rekordu.

45. Dodatkowe pola instancji w rekordzie

```
java

// ❌ BŁĄD KOMPILACJI – rekordy nie mogą mieć dodatkowych pól instancji
record Person(String name) {
    private int age; // błąd!
}

// ✅ Można mieć statyczne pola
record Person(String name) {
    private static final String DEFAULT = "Unknown";
    static int count = 0; // mutowalny statyczny – możliwy, choć niezalecany
}
```

Przyczyna: Wszystkie dane rekordu muszą być jawnie zadeklarowane w nagłówku (header). To jest fundamentalne założenie rekordów — "co widzisz w nagłówku, to wszystkie dane".

Rekord jest **nieodpowiedni** gdy:

1. **Potrzebujesz dziedziczenia** — rekordy są `final`, nie można po nich dziedziczyć.
 2. **Potrzebujesz mutowalnego stanu** — komponenty są `final`.
 3. **Masz złożoną logikę biznesową** — rekordy to DTOs, nie encje domeny.
 4. **Używasz JPA/Hibernate** — frameworki ORM często wymagają bezparametrowego konstruktora i możliwości modyfikacji pól (przez settery lub refleksję).
 5. **Potrzebujesz lazy initialization** pól.
-

47. Niemutowalność rekordów — precyzyjne gwarancje

Rekordy gwarantują **płytką niemutowalność** (shallow immutability):

```
java

record Container(List<String> items) { }

Container c = new Container(new ArrayList<>(List.of("a", "b")));

// Samego rekordu nie można "przepisać" — nie ma setterów
// Ale:
c.items().add("c"); // ⚠️ DZIAŁA! Lista wewnątrz jest mutowalna!
System.out.println(c.items()); // [a, b, c]
```

Co jest gwarantowane: Referencja do listy (pole `items`) nie zmieni się — nie można przypisać nowej listy do `items`. **Co NIE jest gwarantowane:** zawartość obiektów mutowalnych, do których rejestr trzyma referencję.

Prawdziwa głęboka niemutowalność:

```
java

record Container(List<String> items) {
    Container {
        items = List.copyOf(items); // skopiuj i utwórz niemutowalną kopię
    }
}
```

48. Co odróżnia rekord od klasy `final` z polami `final`?

Oba podejścia dają niemutowalność, ale rekordy dodatkowo:

1. **Automatycznie generują** `equals()`, `hashCode()`, `toString()` oparte na komponentach.
2. **Generują akcesory** (`name()` zamiast `getName()`).
3. **Mają semantykę przezroczystości danych** — są traktowane przez JVM i narzędzia jako "czyste nośniki danych".
4. **Uczestniczą w pattern matchingu** (Java 16+).

```
java

// Ręczna klasa final
final class PointManual {
    private final int x, y;
    PointManual(int x, int y) { this.x = x; this.y = y; }
    int x() { return x; }
    int y() { return y; }
    // equals, hashCode, toString — musisz napisać sam
}

// Rekord — automatyczne
record Point(int x, int y) { }
```

49. Rekordy a serializacja JSON (biblioteki)

Starsze biblioteki serializacji JSON (jak niektóre konfiguracje Jacksona bez modułu) mogą mieć problemy z rekordami bo:

- Oczekują bezparametrowego konstruktora (do deserializacji przez odbicie/refleksję).
- Oczekują metod `getX()` zamiast `x()`.
- Rekordy nie mają tych elementów domyślnie.

Jackson z Java 16+ obsługuje rekordy gdy dodasz:

```
xml

<dependency>
  <groupId>com.fasterxml.jackson.module</groupId>
  <artifactId>jackson-module-parameter-names</artifactId>
</dependency>
```

Lub użyjesz adnotacji: `@JsonAutoDetect(fieldVisibility = Visibility.ANY)`.

50. Dlaczego Java nie ma wbudowanego `withXxx()` dla rekordów?

W przeciwieństwie do Kotlin data classes (`copy()`), Java Records nie mają wbudowanej metody `with`.

Powody projektowe:

- Filozofia Javy: minimalizm — biblioteka nie powinna zakładać jak chcesz tworzyć kopie.
- Możesz napisać własną metodę:

```
java

record Person(String name, int age) {
    // Ręczna metoda "with"
    Person withName(String newName) {
        return new Person(newName, this.age);
    }
    Person withAge(int newAge) {
        return new Person(this.name, newAge);
    }
}

Person p = new Person("Alice", 30);
Person older = p.withAge(31); // niemutowalna "modyfikacja"
```

51. Ograniczenia rekordów — współdzielenie stanów

Rekordy nie mogą:

- Dziedziczyć po innych klasach (nie można współdzielić zachowań przez dziedziczenie).
- Mieć niestandardowych pól instancji (tylko komponenty z nagłówka).

Można:

- Implementować interfejsy (współdzielenie zachowań przez interfejsy).
- Mieć wspólną logikę w metodach statycznych (utility methods).
- Używać kompozycji (rekord zawiera inne rekordy jako komponenty).

```
java
```

```
interface HasId { int id(); }

record Product(int id, String name) implements HasId { }
record Order(int id, Product product) implements HasId { }
// Oba współdzielą kontrakt HasId
```

52. Konstruktor kompaktowy — zasady działania

```
java

record User(String email, int age) {
    User {
        // Tutaj możesz:
        // 1. Walidować
        if (age < 0) throw new IllegalArgumentException();
        // 2. Normalizować
        email = email.toLowerCase(); // nadpisanie komponentu – dozwolone
        // 3. NIE możesz dodawać nowych parametrów
        // Po zakończeniu bloku: this.email = email; this.age = age; – niejawnie!
    }
}
```

Co jest prawdą:

- Nie ma listy parametrów — są niejawnie dziedziczone z nagłówka rekordu.
- Przypisanie do pól (`this.email = email`) jest generowane niejawnie na końcu.
- Możesz zmienić wartość przez przypisanie do zmiennej parametru (np. `email = email.toLowerCase()`).
- Jeśli nie ma konstruktora kompaktowego, używany jest domyślny (kanoniczny).

53. Deserializacja a niezmienniki rekordu

Gdy obiekt rekordu jest **deserializowany przez Java Serialization** (`ObjectInputStream`), mechanizm serializacji **omija konstruktor** (używa specjalnej metody JVM). Oznacza to, że walidacja w konstruktorze kompaktowym **NIE jest wywoływana** podczas deserializacji.

```
java
```



```
record SafeRange(int min, int max) {  
    SafeRange {  
        if (min > max) throw new IllegalArgumentException();  
    }  
}  
// ⚠ Deserializacja może odtworzyć obiekt z min > max, omijając walidację!
```

Zabezpieczenie: Zaimplementuj `readObject()` (dla zwykłej serializacji) lub użyj zewnętrznych bibliotek (Jackson, Gson) które respektują konstruktory.

54. Lokalny rekord wewnątrz metody

Od Java 16 rekordy można deklarować lokalnie (wewnątrz metody):

```
java  
  
public void processData(List<String> data) {  
    // Lokalna definicja rekordu  
    record NameLength(String name, int length) { }  
  
    List<NameLength> results = data.stream()  
        .map(s -> new NameLength(s, s.length()))  
        .collect(Collectors.toList());  
  
    results.forEach(nl -> System.out.println(nl.name() + ": " + nl.length()));  
}
```

Interakcja ze zmiennymi otoczenia:

- Lokalny rekord **nie może** bezpośrednio odwoływać się do zmiennych lokalnych metody (w przeciwieństwie do lambda i klas anonimowych).
 - Może używać zmiennych przekazanych jako komponenty lub parametry konstruktora.
 - Jest implicitnie `static` (nie przechwytyje referencji do zewnętrznej klasy).
-

55. Komponent rekordu zawierający mutowalną listę

```
java
```

```

record DataSet(List<Integer> values) { }

List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
DataSet ds = new DataSet(list);

list.add(4);           // ⚠️ zmienia oryginał – ds.values() też widzi zmianę!
ds.values().add(5);    // ⚠️ możliwe – lista jest mutowalna

System.out.println(ds.values()); // [1, 2, 3, 4, 5]

```

Gwarancje: Rekord gwarantuje że `values` zawsze wskazuje na TĘ SAMĄ listę (referencja nie zmieni się). Nie gwarantuje że lista nie zostanie zmodyfikowana.

Obrona w konstruktorze:

```

java

record DataSet(List<Integer> values) {
    DataSet {
        values = List.copyOf(values); // niemodyfikowalna kopia
    }
}

```

56. Nadpisywanie akcesora rekordu

```

java

record Product(String name, double price) {
    // Nadpisanie domyślnego akcesora
    @Override
    public double price() {
        return Math.round(price * 100.0) / 100.0; // zaokrąglenie do 2 miejsc
    }

    // Nadpisanie toString
    @Override
    public String toString() {
        return "Product{" + name + ", " + price() + "}";
    }
}

```

Reguła: Możesz nadpisać dowolny automatycznie generowany metod rekordu: akcesory (`name()`), (`price()`), (`equals()`), (`hashCode()`), (`toString()`). Musisz zachować tę samą sygnaturę.

57. Refleksja i rekordy — pobieranie metadanych

```
java

record Point(int x, int y) { }

Class<?> clazz = Point.class;

// Sprawdzenie czy to rekord
System.out.println(clazz.isRecord()); // true

// Pobranie komponentów rekordu
RecordComponent[] components = clazz.getRecordComponents();
for (RecordComponent rc : components) {
    System.out.println(rc.getName() + ": " + rc.getType().getName());
    // x: int
    // y: int
}

// Wywołanie akcesora przez refleksję
Point p = new Point(1, 2);
Method xMethod = components[0].getAccessor();
System.out.println(xMethod.invoke(p)); // 1
```

`java.lang.reflect.RecordComponent` (Java 16+) umożliwia dynamiczne odkrywanie struktury rekordów.

58. Adnotacje na komponentach rekordu

```
java

record Person(@NotNull @Size(max=50) String name, @Min(0) int age) { }
```

Gdy umieszczasz adnotację na komponencie rekordu, kompilator **propaguje ją** do wszystkich odpowiednich miejsc:

- Pola prywatnego (`private final String name`)
- Parametru konstruktora kanonicznego
- Akcesora (`name()`)

Specyfikacja: Adnotacja jest aplikowana wszędzie tam, gdzie jej `@Target` na to pozwala. Jeśli adnotacja ma `@Target({FIELD, METHOD, PARAMETER})`, trafi do wszystkich trzech.

59. Rekordy a dynamiczne proxy (problemy z dziedziczeniem)

```
java

record Point(int x, int y) { }

// ❌ NIEMOŻLIWE – rekordy są final, nie można tworzyć podklas
// Biblioteki używające CGLIB (Mockito, Spring AOP) nie mogą "otoczyć" rekordu proxy
// bo proxy = podklasa w CGLIB
```

Skutek: Rekordy nie mogą być mockowane przez Mockito (domyślnie używa CGLIB). Można jednak:

- Używać interfejsów (rekordy mogą implementować interfejsy, a interfejsy można mockować).
- Używać Mockito z `mockConstruction()` w nowszych wersjach.

60. Rekord z tablicą — bezpieczeństwo w środowisku wielowątkowym

```
java

record DataArray(int[] values) { }

int[] arr = {1, 2, 3};
DataArray da = new DataArray(arr);

// Wielowątkowy dostęp:
// Wątek A: da.values()[0] = 99; – modyfikacja!
// Wątek B: System.out.println(da.values()[0]); – wyścig danych!
```

Problem:

- Tablica jest mutowalna — wiele wątków może ją jednocześnie modyfikować (race condition).
- Rekord nie synchronizuje dostępu.

Rozwiązania:

```
java
```

```
record DataArray(int[] values) {
    DataArray {
        values = values.clone(); // kopia defensywna – ale wciąż mutowalna!
    }
    // lub
    DataArray(int[] values) {
        this.values = Arrays.copyOf(values, values.length);
    }
}

// Dla pełnej niezmienności użyj List.copyOf() zamiast tablicy
record DataList(List<Integer> values) {
    DataList { values = List.copyOf(values); }
}
```

Nawet po klonowaniu, sama tablica jest mutowalna. Dla środowisk wielowątkowych użyj `Collections.unmodifiableList()` lub `List.copyOf()` zamiast tablic, lub użyj `volatile` / synchronizacji zewnętrznej.